PRISM expansion flow

From a one-off repo scanner to a continuous code-intelligence app. This document covers the target product flow, a four-phase roadmap, and the recommended first slice.

Where PRISM is today

Sign in with GitHub, paste a public or private repository, and PRISM analyzes it on the server: JS/TS and Python parsing with complexity, committed secrets, risky patterns (SQL/shell built from strings, eval, N+1, blocking calls), dependency vulnerabilities via OSV.dev, and a circular-dependency graph. Results become a health score, a 3D repository map and a Claude-written summary, saved per commit in Supabase.

The gap: every analysis is manual, one-off and read-only. Nothing brings people back, and findings cannot be acted on.

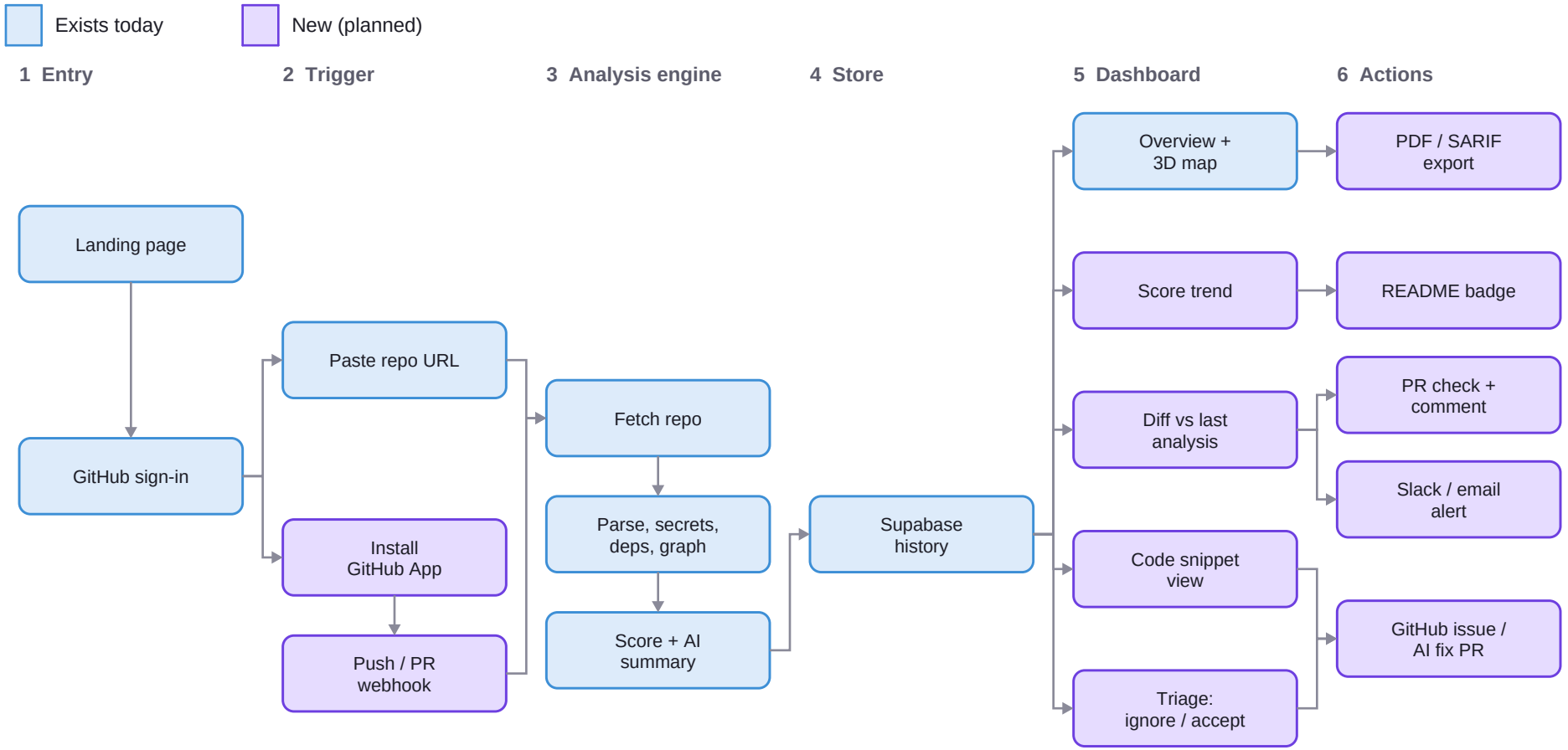
Roadmap at a glance

Phase	Theme	Outcome
1	Make findings actionable	Triage, diff, trend, code snippets. Repeat visits become useful.
2	Make it continuous	GitHub App, PR checks, badges, alerts. PRISM runs without being asked.
3	Use the data	Churn hotspots, repo comparison, org dashboard, exports.
4	Product basics	More languages, repo chat, billing, teams, tests and CI.

Recommended first slice

Triage findings + diff between analyses + score trend chart on the History page. It needs only the Supabase tables PRISM already has (plus one small table for triage state), makes the app useful on repeat visits, and sets up the PR-check work in phase 2.

Target product flow



Phase 1 - Make findings actionable

Fastest to build; uses data PRISM already stores.

Feature	What it does	Where it lands in the code
Triage findings	Mark an issue ignored, false positive or accepted, with a note. Hidden from score and lists.	New table prism_issue_states (user, repo, issue id, state, note); filter in analysis results; src/server/db.ts
Diff between analyses	Compare two commits: new, fixed and unchanged issues.	Both results already in prism_analyses.result; new view in HistoryView.tsx
Score trend chart	Line chart of health score per commit.	health.history in analysisTypes.ts; chart in Overview and History
Code snippet view	Show the offending lines next to each finding.	Fetch file at commit on demand via src/server/github.ts

Phase 2 - Make it continuous

What teams adopt a tool for. Needs a GitHub App instead of OAuth only.

Feature	What it does	Where it lands in the code
GitHub App + webhooks	Analyze automatically on every push and pull request.	New route src/app/api/github/webhook; verify signature; queue the analysis
PR check + comment	Post 'Health 82 to 76, 2 new critical issues' on the PR.	Uses the diff from phase 1; Checks API via the App token
README badge	SVG score badge for any analyzed repo.	New route src/app/api/badge/[owner]/[repo]
Alerts	Slack or email when the score drops or a secret is committed.	Per-user settings page; rules on score delta and secret findings

Phase 3 - Use the data

Feature	What it does	Where it lands in the code
Churn x complexity hotspots	Rank files that change often and are complex.	Fill hotspots[].churn and fanIn from git history in analysis/index.ts
Compare repos / branches	Side-by-side scores and issue categories.	New /app/compare view reading prism_analyses
Org dashboard	All repos for a user or org on one page.	Extend /app/repos with aggregate stats
Export	PDF, Markdown and SARIF so findings appear in GitHub's Security tab.	New export route; SARIF mapping from Issue type

Phase 4 - Product basics

Feature	What it does	Where it lands in the code
More languages	Go, Java, Rust; a real Python AST instead of line-by-line.	New parsers beside src/server/analysis/parse.ts
Ask your repo	Chat grounded in analysis facts, with issue citations.	Extend src/server/ai.ts; UI in AI Review section
Pricing + billing	Free vs paid limits (private repos, auto-analysis); Stripe.	Pricing page, subscription table, limit checks in loadRepo.ts
Teams + share links	Read-only share links for a report; team access.	New share-token table and route
Settings, onboarding, errors	Empty states, rate-limit handling, settings page.	src/app/app/account and RepoError.tsx
Tests + CI	No tests exist yet; detectors are easy to regress.	Add vitest for parse.ts, secrets.ts, graph.ts and a GitHub Actions workflow

Open decisions

1. Start with the recommended first slice, or jump straight to the GitHub App?
2. Free tier shape: how many private-repo analyses before billing applies?
3. Where do alerts go first: Slack, email, or both?